

AI Mapping Copilot - Spec Packet

AI Mapping Copilot — Spec Packet

Live App: <https://csv-mapping-uniblox-production.up.railway.app>

GitHub: <https://github.com/abhi-anand-07/csv-mapping-uniblox>

1. Problem & Goals

Problem

Insurance operations teams receive spreadsheets from brokers, employers, and third parties. Every sender uses different column names for the same data:

Sender A	Sender B	Canonical
Emp ID	employee#	employee_id
F Name	given name	first_name
DOB	Birth Dt	date_of_birth
Annual Pay	compensation	annual_salary

Manual mapping is slow, error-prone, and doesn't scale.

Goals

- Automate column-header normalization to a canonical schema
 - Explain AI reasoning so users trust proposals
 - Require human approval before publishing clean data
 - Handle messy real-world CSVs (abbreviations, typos, missing headers)
-

2. User Workflow

Upload CSV → AI Proposes Mappings → User Reviews & Edits → Approve → Download Clean CSV

1. **Upload** — User drops a CSV into the browser

2. **AI Mapping** — Multi-step agent parses headers, sample rows, calls Gemini LLM, validates output
3. **Review** — User sees a table with confidence scores, reasoning, warnings. Can edit any mapping via dropdown.
4. **Approve** — User confirms mappings. Publish blocked if required fields are missing.
5. **Download** — Backend generates canonical CSV with correct column names and order.

3. Requirements

Functional

ID	Requirement	Status
FR-1	Accept CSV upload via drag-and-drop	✓
FR-2	Parse CSV with encoding fallback (utf-8, latin1, cp1252)	✓
FR-3	Propose mappings to 12-field canonical schema	✓
FR-4	Display confidence score (0–1) and level (High/Medium/Low/Uncertain)	✓
FR-5	Show human-readable reasoning per mapping	✓
FR-6	Allow inline editing of mappings before approval	✓
FR-7	Block publish until required fields are mapped	✓
FR-8	Generate downloadable canonical CSV on publish	✓
FR-9	Provide sample CSV for testing	✓

Non-Functional

ID	Requirement	Status
NFR-1	Only headers + 5 sample rows sent to LLM (no raw data)	✓
NFR-2	Works without LLM API key (rule-based fallback)	✓
NFR-3	Max file size: 10MB	✓
NFR-4	Response time: < 5s for mapping proposal	✓

4. AI Behavior + Guardrails

Multi-Step Agent Pipeline

1. **Ingestion** — Extract headers, sample rows, detect data types (string, numeric, date, boolean, state, zip)
2. **Mapping** — Gemini LLM proposes mappings with confidence and reasoning via structured JSON prompt
3. **Validation** — Detect conflicts (multiple sources → same target), flag low confidence, check required field coverage
4. **Explanation** — Generate summary paragraph + per-column reasoning

Guardrails

Guardrail	Implementation
No raw data to LLM	Only headers + 5 rows sent to Gemini
Human-in-the-loop	All mappings require explicit user approval
Required fields	Publish blocked until employee_id, first_name, last_name, date_of_birth are mapped
Confidence display	Every mapping shows visual badge + percentage
Conflict detection	Warning badge when multiple columns map to same target
Fallback mode	Rule-based mapper with 100+ header aliases works without API key
Edit audit	User edits override AI proposals with 100% confidence

5. Evaluation Plan

Automated Tests

- **Unit:** CSV parser handles unnamed columns, duplicates, encoding issues
- **Unit:** Type detector correctly identifies boolean, date, numeric, state, zip
- **Integration:** Full upload → map → edit → approve → publish → download flow

Manual Evaluation

Test Case	Input	Expected
Clean headers	employee_id, first_name...	100% confidence, all fields mapped
Messy headers	Emp#, FirstNm, DOB, St	Correct mapping with reasoning
Ambiguous columns	date hired + start date	Conflict warning on hire_date

Test Case	Input	Expected
Missing required	No date_of_birth column	Missing required alert, publish blocked
Non-CSV file	Upload .xlsx	Rejected with clear error

Success Metrics

- **Precision:** % of AI-proposed mappings that are correct (target: > 90%)
- **Coverage:** % of required fields found without user edit (target: > 85%)
- **Time-to-map:** Median seconds from upload to proposal (target: < 3s)

6. Instrumentation & Logging

Layer	What is Logged
Backend	Session ID, file name, row count, proposed mappings, user edits, approval status, publish metadata
AI Agent	Gemini prompt tokens, response time, fallback usage, parsing warnings
Frontend	Upload events, edit actions, approval clicks, download events

Privacy

- Full CSVs stored server-side only during active session
- No proprietary data sent to external LLM (only headers + 5 anonymized rows)
- Sessions are in-memory; no persistent database of user uploads

7. Rollout Plan

Phase 1: Internal Validation (Week 1)

- Team tests with 10 real-world messy CSVs
- Fix edge cases in parser and fallback mapper
- Tune Gemini prompt based on failure modes

Phase 2: Beta Users (Week 2–3)

- 5 operations team members get access
- Collect feedback on confidence thresholds and edit UX
- Add aliases based on domain-specific headers discovered

Phase 3: General Availability (Week 4)

- Open to all operations staff
- Monitor precision/coverage metrics
- Add SSO and audit logging for compliance

Appendix: Tech Stack & Deployment

Layer	Tech
Frontend	React 18, TypeScript, Vite
Backend	Python, FastAPI, Pandas
AI	Google Gemini 3.1 Flash Lite
Hosting	Railway (Docker, single container)
Repo	https://github.com/abhi-anand-07/csv-mapping-uniblox
Live URL	https://csv-mapping-uniblox-production.up.railway.app